FINANCIAL ENGINEERING

KBP FFÚ & KSTP FIS

Case Studies in Capital Markets and Banking

Multi-factor models

Ing. Milan Ficura, Ph.D.
milan.ficura@vse.cz

EVROPSKÁ UNIE
Evropské strukturální a investiční fondy
Operační program Výzkum, vývoj a vzdělávání

MINISTERSTVO ŠKOLSTVÍ, MLÁDEŽE A TĚLOVÝCHOVY

Section 4:
Multi-factor models

Portfolio optimization and CAPM

- **Multi-Factor models**
- Fama-French data library
- Multi-Factor models in practice

CAPM model

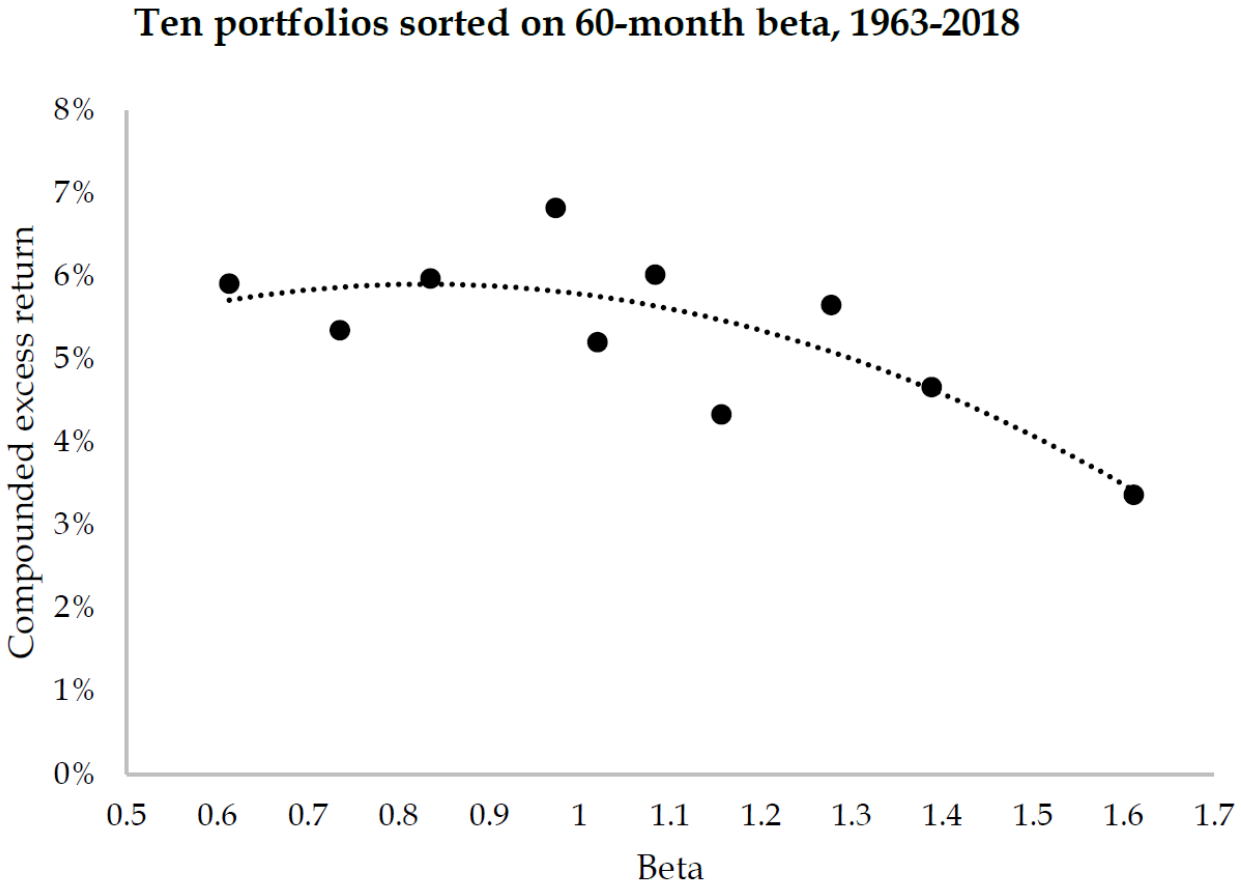
- Treynor (1961), Sharpe (1964), Lintner (1965), Mossin (1966)
- Solves the problem of asset pricing in the MPT framework
- Equilibrium asset returns depend on their contribution to the total risk of the market portfolio
- Historical asset returns can be decomposed as:

$$R_{i,t} - R_f = \alpha_i + \beta_i \left[R_{M,t} - R_f \right] + \epsilon_{i,t}$$

- Where $R_{M,t}$ is the market portfolio return and $\epsilon_{i,t} = N(0, \sigma_i)$
- The total variability of $R_{i,t} - R_f$ is thus decomposed into:
  - **Systematic risk**, equal to β_i, i.e. the sensitivity to $\left[R_{M,t} - R_f \right]$ (risk factor)
  - **Specific risk**, equal to σ_i (which can be diversified away)
- Investors recieve risk premium only for **systematic risk**
- **Expected returns** of any asset are thus given by the equation:

$$\mathrm{E}(R_i) = R_f + \beta_i \left[\mathrm{E}(R_M) - R_f \right]$$

Empirical testing of CAPM (1)

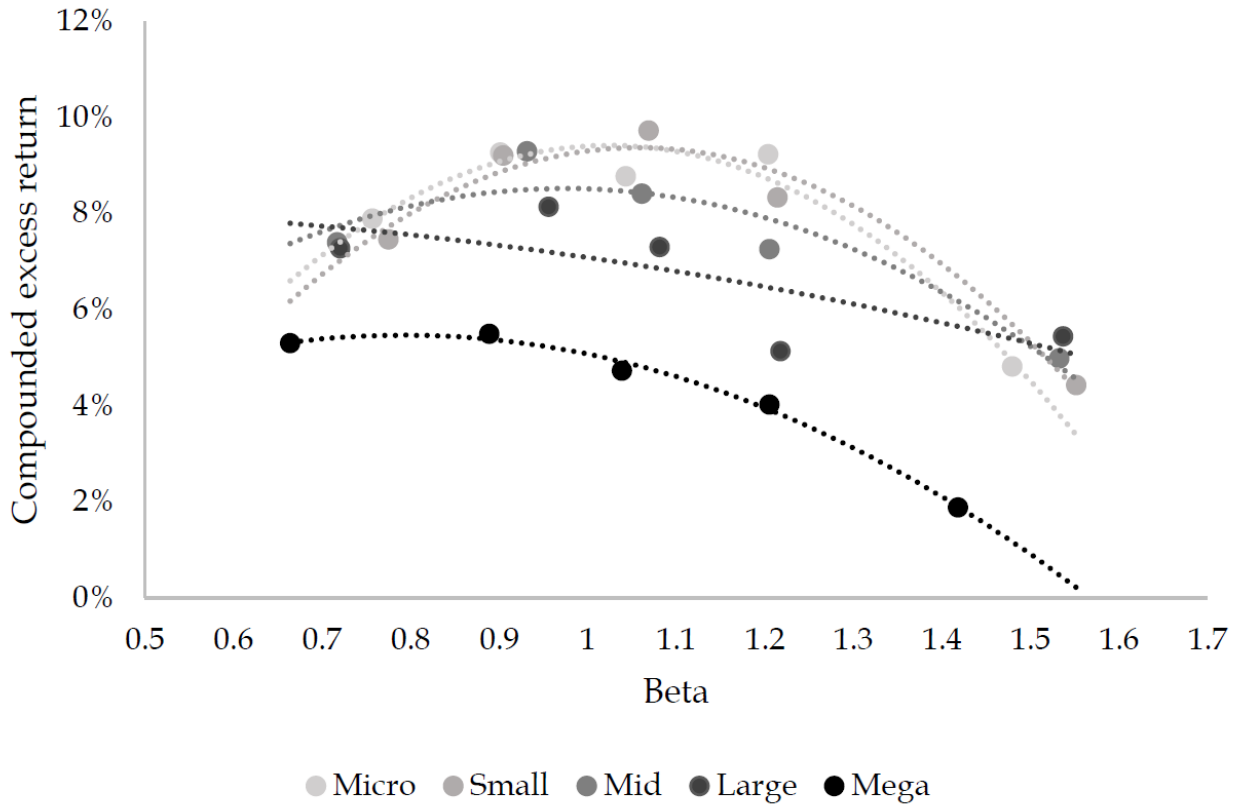


Source: Blitz, D., Vliet, P., Baltussen, G. (2019). The Volatility Effect Revisited

- Based on empirical testing CAPM does not seem to work well
  - Can partially be explained by the **effect of other factors**
  - Market risk premium is not the only risk premium

Empirical testing of CAPM (2)

5x5 portfolios sorted on size and 60-month beta, 1963-2018



Source: Blitz, D., Vliet, P., Baltussen, G. (2019). The Volatility Effect Revisited

- **CAPM** seems to work for some of the sub-groups with beta up to 1.1
- With additional factors the relationship becomes more linear, but is still too flat for low beta and high beta stocks – **volatility puzzle**

APT model (Ross, 1976)

- Arbitrage Pricing Theory (APT) model
  - Goal is to explain cross-sectional stock returns better than CAPM
  - Adds additional risk-factors to the model
- The result is a **multi-factor model** with factor-specific betas

$$R_{i,t} = \alpha_i + \beta_{i,1}F_{1,t} + \beta_{i,2}F_{2,t} + \cdots + \beta_{i,n}F_{n,t} + \varepsilon_{i,t}$$

- Where $F_{j,t}$ denotes the j-th systematic factor and $\beta_{i,j}$ is the sensitivity of stock i to factor j
- The expected stock return is then given as:

$$E(R_{i,t}) = R_{f,t} + \beta_{i,1}RP_{1,t} + \beta_{i,2}RP_{2,t} + \cdots + \beta_{i,n}RP_{n,t}$$

- Where $RP_{j,t}$ is the expected return of factor j at time t
- Served as a framework for the development of other multi-factor models (Fama-French model, Chen, Roll and Ross model, etc.)

Chen-Roll-Ross macro model (1986)

- **Chen-Roll-Ross macro model (1986)** uses macroeconomic factors to predict future stock returns:
  1. **Industrial production**
     - Reflects changes in cash-flow expectations
  2. **Yield spread betwen high risk vs. low risk corporate bonds**
     - Reflects changes in risk preferences
  3. **Difference between short-term vs. long-term interest rates**
     - Reflects shifts in time preferences
  4. **Unanticipated inflation**
  5. **Expected inflation**
- Factor risk premiums - derived with **Fama-MacBeth 2-Stage method**:
  A. Use time-series regressions to find **stock sensitivities** to the factors:

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_{j,i}\, F_{j,t} + \varepsilon_{i,t}$$

  B. Use cross-sectional data to obtain the **factor risk premiums**:

$$R_{i,t} = \alpha_t + RP_{j,t}\beta_{j,i} + \epsilon_{i,t}$$

Fama-French 3-factor model (1993)

- **Fama and French (1993)** extension of the CAPM model, which adds two additional factors:
  - **Size effect –** Returns of small-cap stocks vs. large-cap stocks
  - **Value effect** – Return of stocks with high Book-to-Market (B/P) ratio vs. the return of stocks with low B/P ratio.
- The model equation for stock returns is:

$$E(R_{i,t}) = R_{f,t} + \beta_i[E(R_{m,t}) - R_{f,t}] + \beta_{S,i}SMB_t + \beta_{V,i}HML_t$$

- Where:
  - SMB_t = expected return of small-cap stocks minus large-cap stocks
  - HML_t = expected return of high B/P stocks minus low B/P ratio
- Problematic interpretation of the HML_t factor
  - Is it a risk-factor or a violation of the medium form of market efficiency?

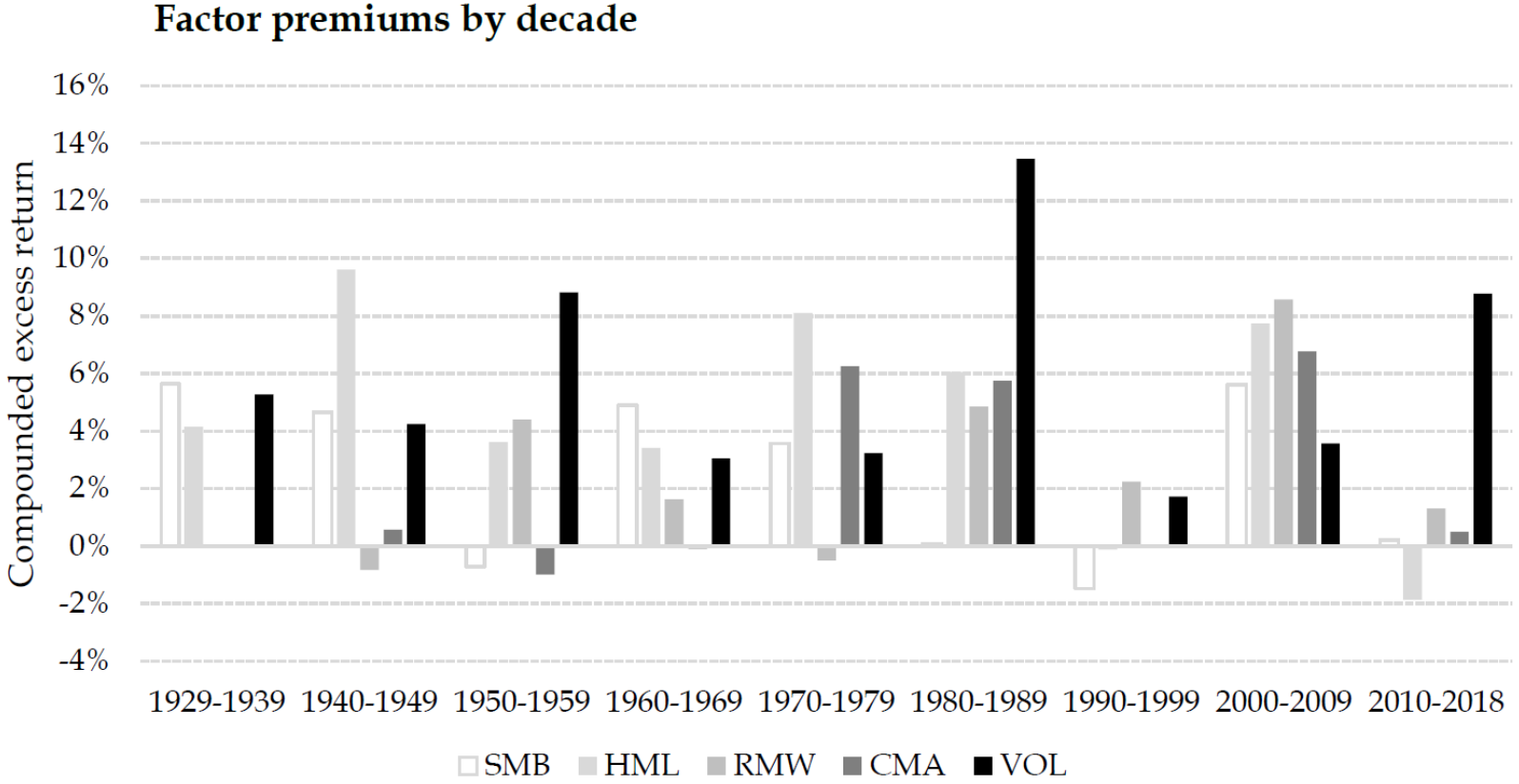
Carhart 4-factor model (1997)

- **Carhart model (1997)** extends the Fama-French three factor model with the momentum effect:
  - **Momentum effect –** Returns of past winners (over the previous 12 months) vs. returns of past losers
- The model equation for stock returns is:

$$E(R_{i,t}) = R_{f,t} + \beta_i[E(R_{m,t}) - R_{f,t}] + \beta_{S,i}SMB_t + \beta_{V,i}HML_t + \beta_{M,i}UMD_t$$

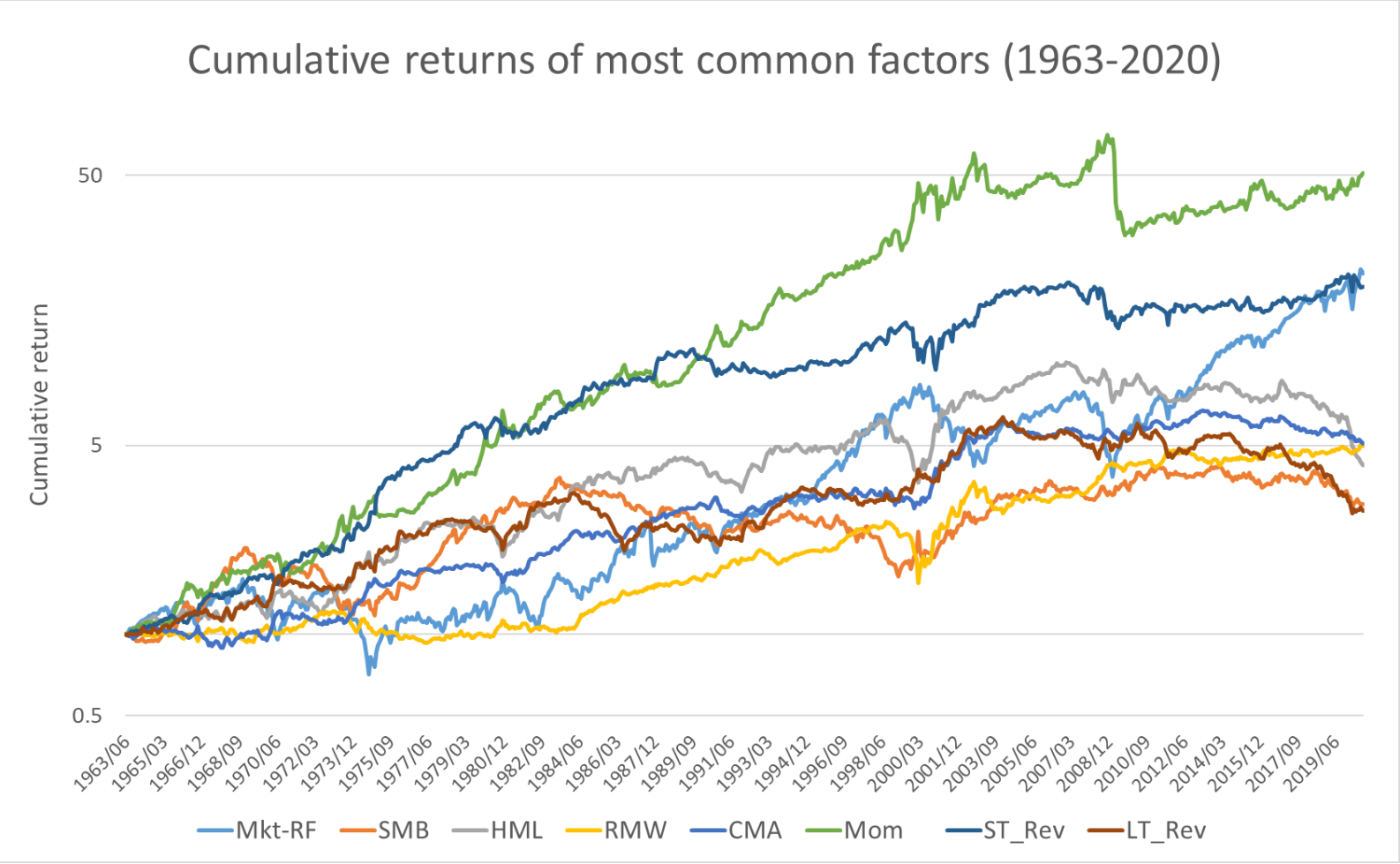
- Where:
  - UMD_t = monthly premium of the previous 12-month winners to the previous 12-month losers
- Interpretation of UMD_t is even more problematic:
  - Is it a risk-factor or a violation of the weak-form of market efficiency?

Factor premiums by decade (US)



- SMB_t = return of small-cap stocks minus large-cap stocks
- HML_t = return of high B/P stocks minus low B/P ratio stocks
- RMV_t = return of most profitable vs. least profitable companies
- CMA_t = return of aggressively vs. conservatively investing firms
- VOL_t = return of least volatile vs. most volatile stocks

Cumulative factor returns



Returns of most commonly used risk-factors (period from 1963-2020, monthly rebalancing, logarithmic scale)

The factors are: **Mkt-Rf** (market return vs. risk-free rate), **SMB** (small vs. big), **HML** (high B/V vs. low B/V), **RMW** (profitable vs. unprofitable), **CMA** (aggresivelly vs. conservativelly investing), **Mom** (high vs. low return over last 12-2 months), **ST_Rev** (low vs. high return over last 1 month), **LT_Rev** (low vs. high return over last 60-13 months)

Data source: https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html

Portfolio optimization and CAPM

- ✓ Multi-Factor models
- ➢ **Fama-French data library**
- ❖ Multi-Factor models in practice

Fama-French data library (1)

- **Fama-French data library:**
- Respository of returns of portfolios (quantile portfolios, factor-mimicking portfolios, sector portfolios, etc.) used in some of the Fama & French papers
- The library is regularly updated and can be accessed on the website: http://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html#Research

- **R package frenchdata:**
- Allows us to access datasets from the Fama-French data library directly from R
- Available from CRAN: https://cran.r-project.org/web/packages/frenchdata/index.html

- **Installing and loading the frenchdata package:**

```
# Installing the frenchdata R package
install.packages("frenchdata")
# Loading the frenchdata R package
library(frenchdata)
```

- **Show the available datasets:**

```
# French library datasets
data_sets <- get_french_data_list()
View(data_sets$files_list)
```

Fama-French data library (2)

- **Fama-French library datasets:**
- The library currently contains **297 datasets** of portfolio returns
- Illustration of some of the datasets related to the **US market:**

	name	file_url
1	Fama/French 3 Factors	ftp/F-F_Research_Data_Factors_CSV.zip
2	Fama/French 3 Factors [Weekly]	ftp/F-F_Research_Data_Factors_weekly_CSV.zip
3	Fama/French 3 Factors [Daily]	ftp/F-F_Research_Data_Factors_daily_CSV.zip
4	Fama/French 5 Factors (2x3)	ftp/F-F_Research_Data_5_Factors_2x3_CSV.zip
5	Fama/French 5 Factors (2x3) [Daily]	ftp/F-F_Research_Data_5_Factors_2x3_daily_CSV.zip

- **Fama/French 3 Factors** → Factor-Mimicking portfolios used in the 3-factor Fama-French model ($Mkt_t - Rf_t$ = Market return minus the risk-free rate, SMB_t = return of small minus big firms, HML_t = Return of firms with high vs. small Book-to-Market rato). The portfolios are constructed with **monthly rebalancing**
- **Fama/French 3 Factors [Weekly]** → Factor-Mimicking portfolios used in the 3-factor Fama-French model, but with **weekly rebalancing**
- **Fama/French 3 Factors [Daily]** → Factor-Mimicking portfolios used in the 3-factor Fama-French model, but with **daily rebalancing**
- **Fama/French 5 Factors (2x3)** → Factor-Mimicking portfolios used in the 5-factor Fama-French model, which contains in addition to the 3-factor model also the RMV_t (return of most vs. least profitable firms) and CMA_t factors (return of most conservativelly vs. most aggresivelly investing firms) (with **monthly relabancing**).

Fama-French data library (3)

- **Download selected dataset from the F-F library:**
- In order to download a dataset from the Fama-French library, we can use the code:

```
# Load Fama-French factors (daily)
ff_5_factors <- download_french_data("DATASET_NAME")
```

- For example, to get the returns of the **factor-mimicking portfolios** used in the **5-factor Fama-French model** with **daily rebalancing**, we can use the code:

```
# Load Fama-French factors (daily)
ff_5_factors <- download_french_data("Fama/French 5 Factors (2x3) [Daily]")
```

- To extract the data series, we can then run:

```
# Extract factors
factors_ff5 <- ff_5_factors$subsets$data[[1]]
```

- The first rows of the dataset look as follows:

	date	Mkt-RF	SMB	HML	RMW	CMA	RF
1	19630701	-0.67	0.01	-0.35	0.03	0.11	0.012
2	19630702	0.79	-0.31	0.24	-0.08	-0.25	0.012
3	19630703	0.63	-0.16	-0.09	0.13	-0.24	0.012

Note: Daily rebalancing means that the stocks on the US market (NYSE+Nasdaq+AMEX) are sorted at the end of each day based on a given characteristics, long-short quantile portfolios are formed, and their return in the next day is computed, giving us the value of the factor

- Where **Mkt-RF** corresponds to the market factor, **SMB** to the size factor (small minus big), **HML** to the value factor (high vs. low B/M firms), **RMW** to the profitability factor, **CMA** to the investment factor, and **RF** to the risk-free rate
- The returns are given as **compound returns** expressed in **percentage points**

Factor-Mimicking portfolio returns

- **Task:** Using the datasets in the Fama-French data library, compute the cumulative returns of the **Mkt-RF, HML, SMB, RWA** and **CMA** portfolios from the 5-Factor Fama-French model (using **daily rebalancing**)
- **Loading the data:**

```
# Loading the frenchdata R package
library(frenchdata)
# Load Fama-French factors (daily)
ff_5_factors <- download_french_data("Fama/French 5 Factors (2x3) [Daily]")
# Extract factors
factors <- ff_5_factors$subsets$data[[1]]
```

- **Convert returns to percent:**

```
# Convert returns to percent
as_percent = function(x) {x/100}
factors2 <- factors %>%
  mutate_at(vars(-date),as_percent)
# Convert date to date format
factors2 <- factors2 %>%
  mutate(date = as.Date(as.character(date), "%Y%m%d"))
# Convert compound returns to log-returns
as_logreturns = function(x) {log(1+x)}
factors3 <- factors2 %>%
  mutate_at(vars(-date),as_logreturns)
```

Cumulative compound returns (2)

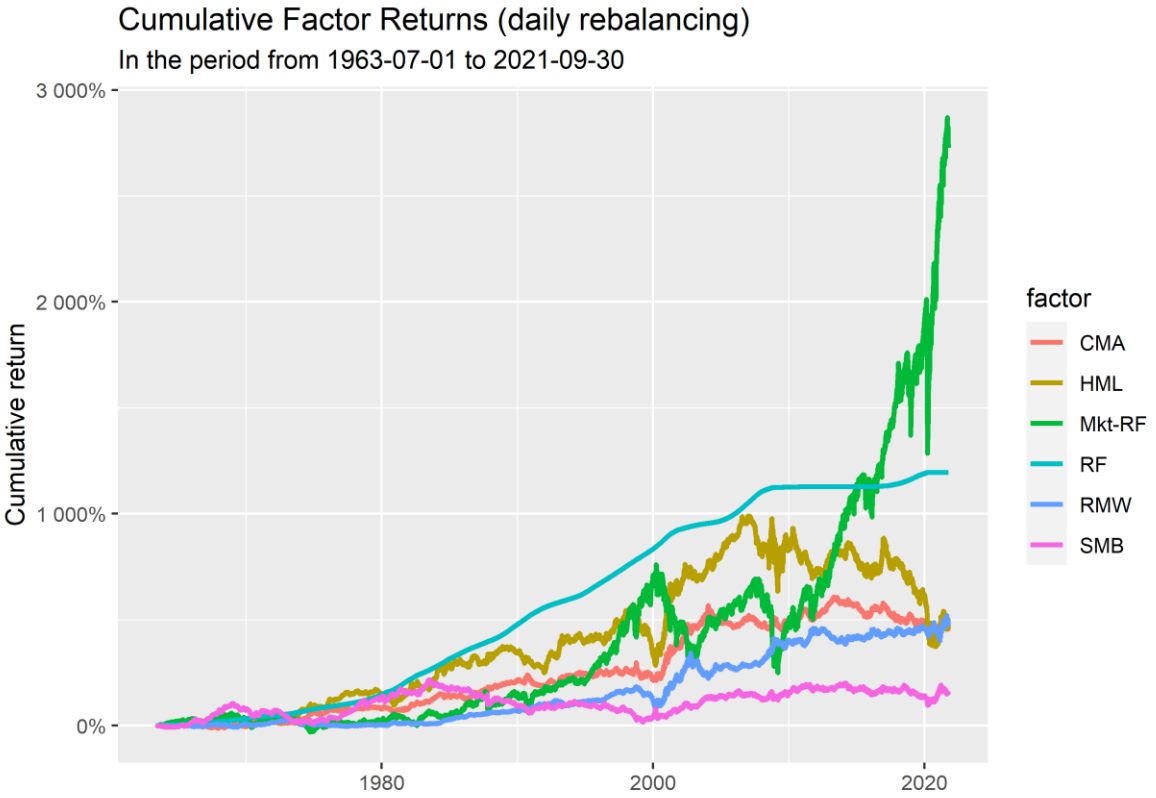
- **Compute cumulative compound returns:**

```
# Convert to long format
lfactors <- factors2 %>%
  gather(factor,return,-date)
# Compute cumulative factor returns in long format
lfactors <- lfactors %>%
  group_by(factor) %>%
  mutate(cumret = cumprod(1+return)-1)
```

- **Illustration of the returns evolution:**

```
# Data range
tStart <- min(lfactors$date)
tEnd <- max(lfactors$date)
# Cumulative logarithmic returns
c00 <- lfactors %>%
  group_by(factor) %>%
  ggplot(aes(x = date, y = cumret, color = factor)) +
  geom_line(size = 1) +
  labs(title = paste0("Cumulative Factor Returns (daily rebalancing)"),
       subtitle = paste0("In the period from ",tStart," to ",tEnd),
       x = "", y = "Cumulative return") +
  scale_y_continuous(labels = scales::percent)
```

Cumulative compound returns (1)



- All of the Factor-Mimicking Portfolios achieved positive returns in the period since 1963
- Take note that all of the Factor-Mimicking Portfolios are constructed as **self-financing** (i.e. containing equal volume of long and short positions), and therefore, if the given factor was not priced by the market, the expected return of these portfolios would be equal to zero
- As the long-term returns of the portfolios are larger than zero, it seems that the market prices the systematic risk arising from the sensitivity to the Fama-French factors

Cumulative logarithmic returns (1)

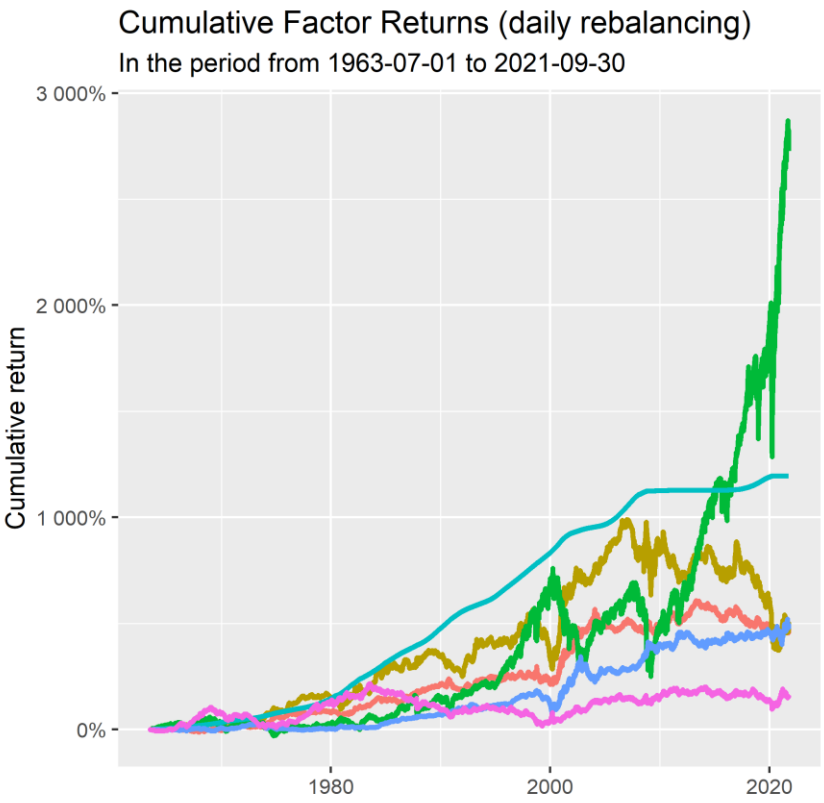
- **Compute cumulative logarithmic returns:**

```
# Convert to long format
lfactors <- factors3 %>%
  gather(factor,return,-date)
# Compute cumulative factor returns in long format (logarithmic)
lfactors <- lfactors %>%
  group_by(factor) %>%
  mutate(cumlogret = cumsum(return))
# Compute cumulative factor returns in long format (exponential)
lfactors <- lfactors %>%
  group_by(factor) %>%
  mutate(cumret = exp(cumlogret)-1)
```

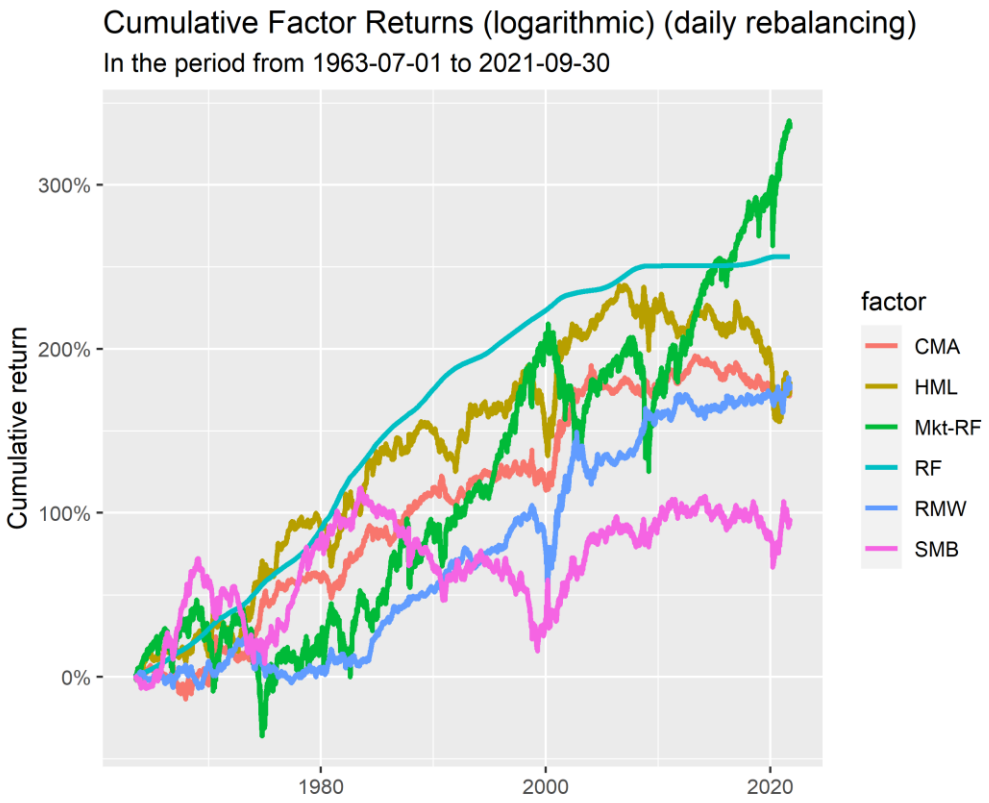
- **Note:** In financial econometrics it is more common to work with **logarithmic returns**, which can be summed up to get the linearized evolution of the cumulative portfolio returns
- The usage of logarithmic returns allows us also to easily transform the portfolio evolution back to the **compound** (**exponential**) returns as shown in the code above

Cumulative logarithmic returns (2)

❖ **Compound returns:**



Logarithmic returns:



❖ Exponential trend of the compound returns gets transformed into a linear trend in the case of the logarithmic returns

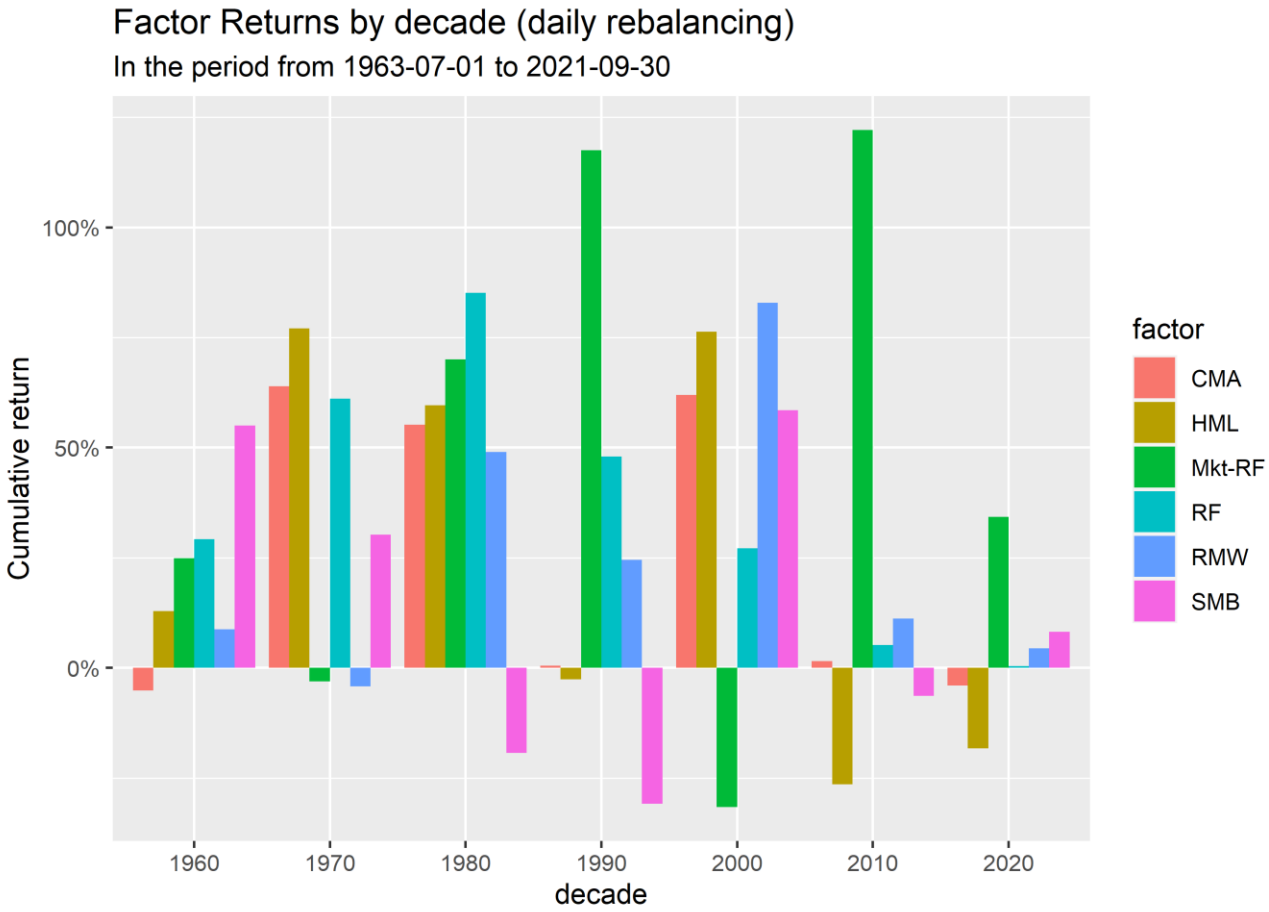
❖ We can further see that while the Fama-French factors oprovided high returns in earlier decades, the **HML**, **CMA** and **SMB** portfolios have actually earned a **negative return over the last decade**

Logarithmic returns per decade (1)

- **Compute logarithmic returns by decade:**

```
# Logarithmic returns by decade
dfactors <- lfactors %>%
  mutate(year = year(date),
         decade = floor(year/10)*10) %>%
  group_by(factor,decade) %>%
  summarize(return = sum(return))
# Plot annual returns
dfactors %>%
  ggplot(aes(x = as.character(decade), y = return, fill = factor)) +
  geom_bar(stat="identity", position = "dodge") +
  labs(title = paste0("Factor Returns by decade (daily rebalancing)"),
       subtitle = paste0("In the period from ",tStart," to ",tEnd),
       x = "decade", y = "Cumulative return") +
  scale_y_continuous(labels = scales::percent)
```

Logarithmic returns per decade (2)



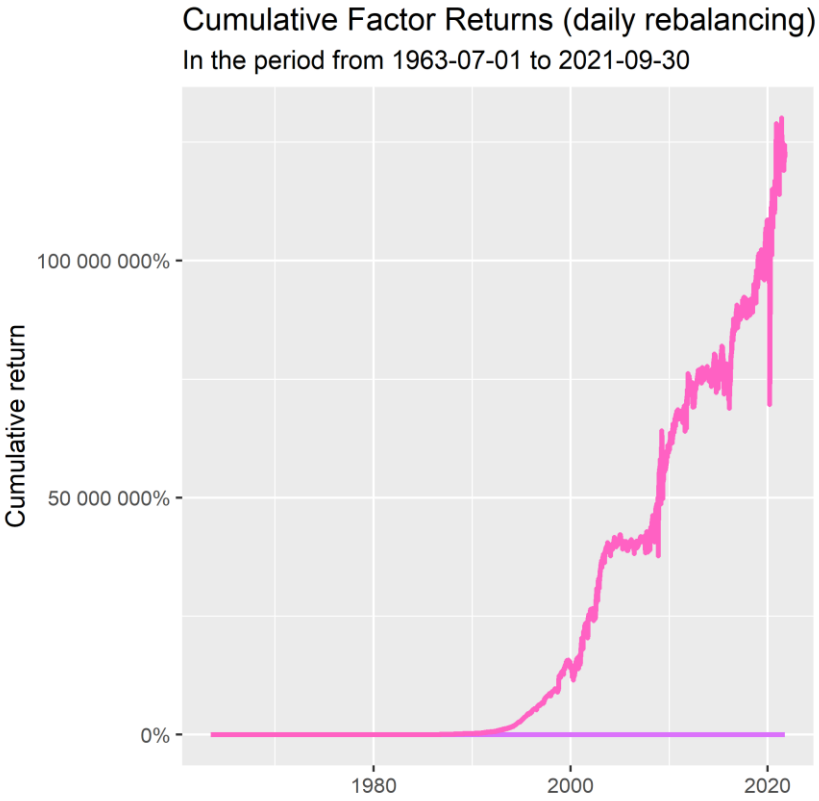
- We can see that in the last two decades **(2000-2009** and **2010-2019)**, only the **market factor** (**Mkt-RF**) and the **profitability factor (RMW)** (to a lesser degree) achieved positive returns
- It is unclear whether the negative performance of the other factors is only temporary, or whether the factors are not priced by the market anymore

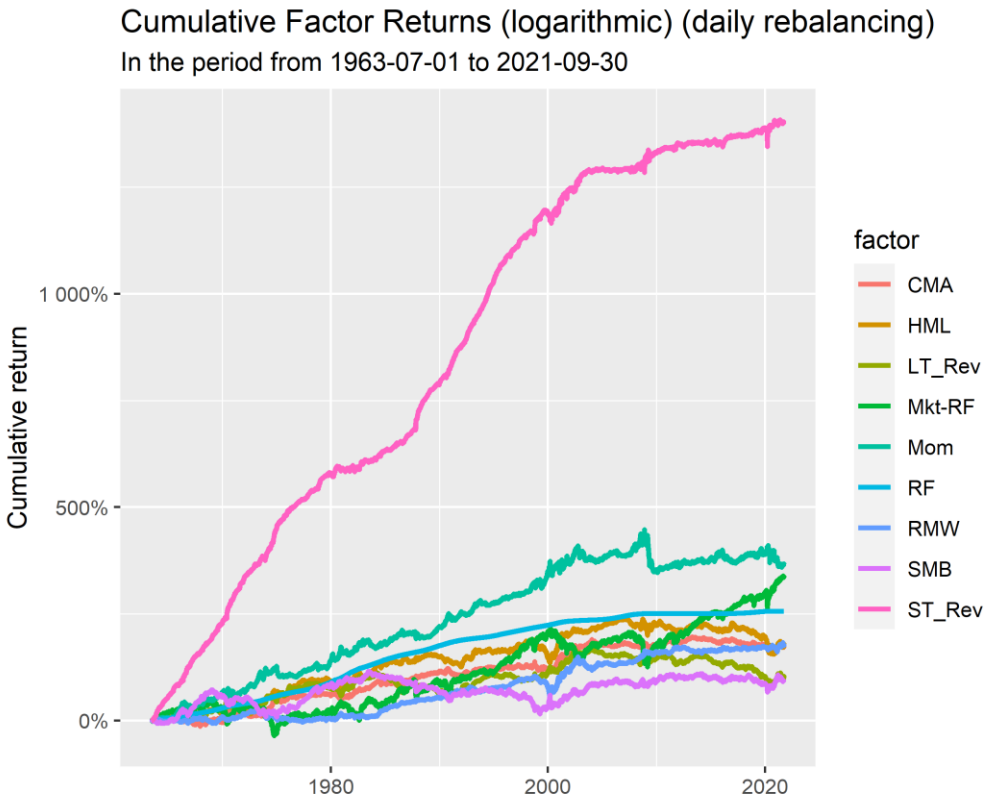
Factor-Mimicking portfolio returns

- **Task:** Extend the results in order to also contain the following factors: **Mom** (momentum), **ST-Rev** (short-term reversion), and **LT-Rev** (long-term reversion)
- **Where:**
  - **Mom** = Portfolio of stocks that grew the most vs. least in the previous 12 months (minus the last month)
  - **ST-Rev** = Portfolio of stocks that grew the least vs. the most in the last 1 month
  - **LT-Rev** = Portfolio of stocks that grew the least vs. the most in the last 5 years (minus the last year)
- **Solution:**
- We just need to modify the loading of the dataset as follows:

```
# Load factors (daily)
ff_5_factors <- download_french_data("Fama/French 5 Factors (2x3) [Daily]")
mom_factors <- download_french_data("Momentum Factor (Mom) [Daily]")
strev_factors <- download_french_data("Short-Term Reversal Factor (ST Rev) [Daily]")
ltrev_factors <- download_french_data("Long-Term Reversal Factor (LT Rev) [Daily]")
# Extract factors
factors_ff5 <- ff_5_factors$subsets$data[[1]]
factors_mom <- mom_factors$subsets$data[[1]]
factors_strev <- strev_factors$subsets$data[[1]]
factors_ltrev <- ltrev_factors$subsets$data[[1]]
# Join factors from different datasets
factors <- factors_ff5 %>%
  left_join(factors_mom,by="date") %>%
  left_join(factors_strev,by="date") %>%
  left_join(factors_ltrev,by="date")
```

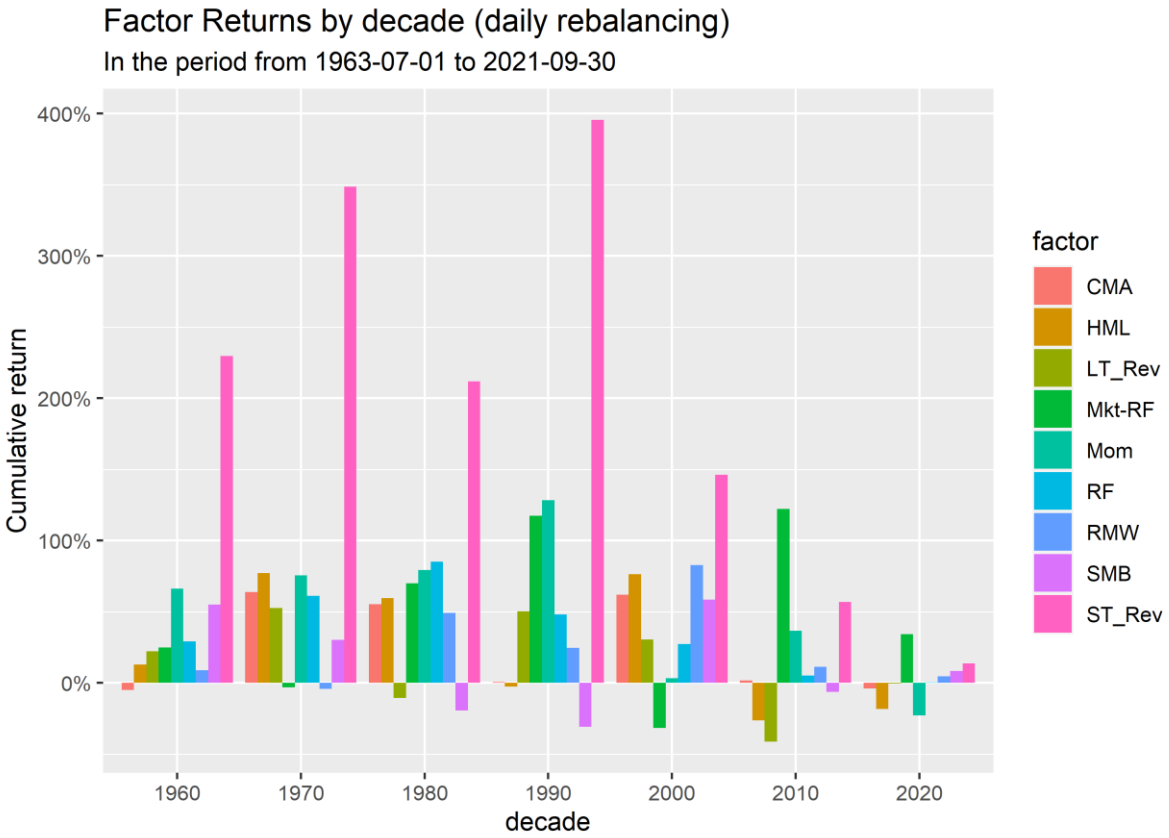
Factor-Mimicking portfolio returns





- We can see that the **Short-term reversion (ST_Rev) factor** dominates all of the other factors, achieving extremely large compound cumulative returns
- **Note:** These results, are, howere, **misleading**, as in practice the **ST_Rev** Factor-Mimicking portfolio with **daily rebalancing**, would exhibit **extremely large portfolio turnover**, and thus be subject to **very large transaction costs** (i.e. on a daily basis we would need to buy the stocks that fell the most and short the ones that grew the most over the last month)

Factor-Mimicking portfolio returns



- We can also see that inspite of its extreme performance in the past, even the **ST_Rev** factor seems to perform not so well over the last 2 decades (especially in **2000-2009**)
- This could be because of the gradual decrease of transaciton costs in the last 2 decades, which allowed investors to better utilize the short-term mean-reversion effects on the market, making the markets more efficient with respect to these effects

Factor-Mimicking portfolio returns

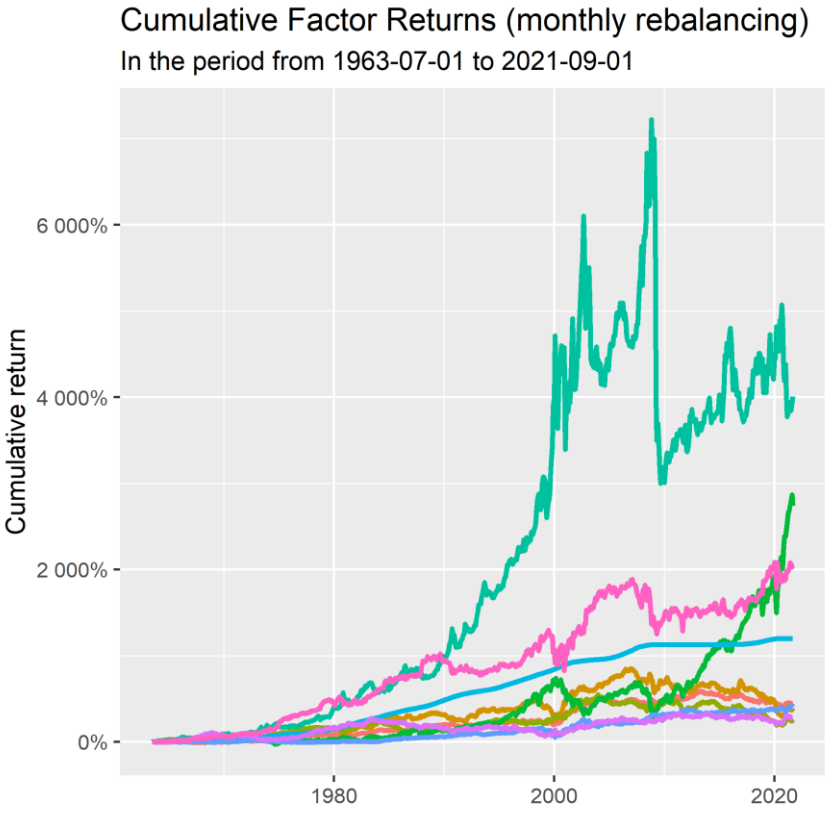
- **Task:** Recalculate the resutls for the portfolios with **monthly rebalancing**
- **Note:** While **daily rebalancing** is usefull to extract the theoretical evolution of the unobservable factors through the Factor-Mimicking Portfolio methodology, for the purposes of assessing the actual (practically achievable) returns of the Factor-Mimicking portfolios it is more appropriate to work with **monthly rebalancing**, as it is much less sensitive to transaction costs
- **Solution:**
- We just need to modify the loading of the dataset as follows:

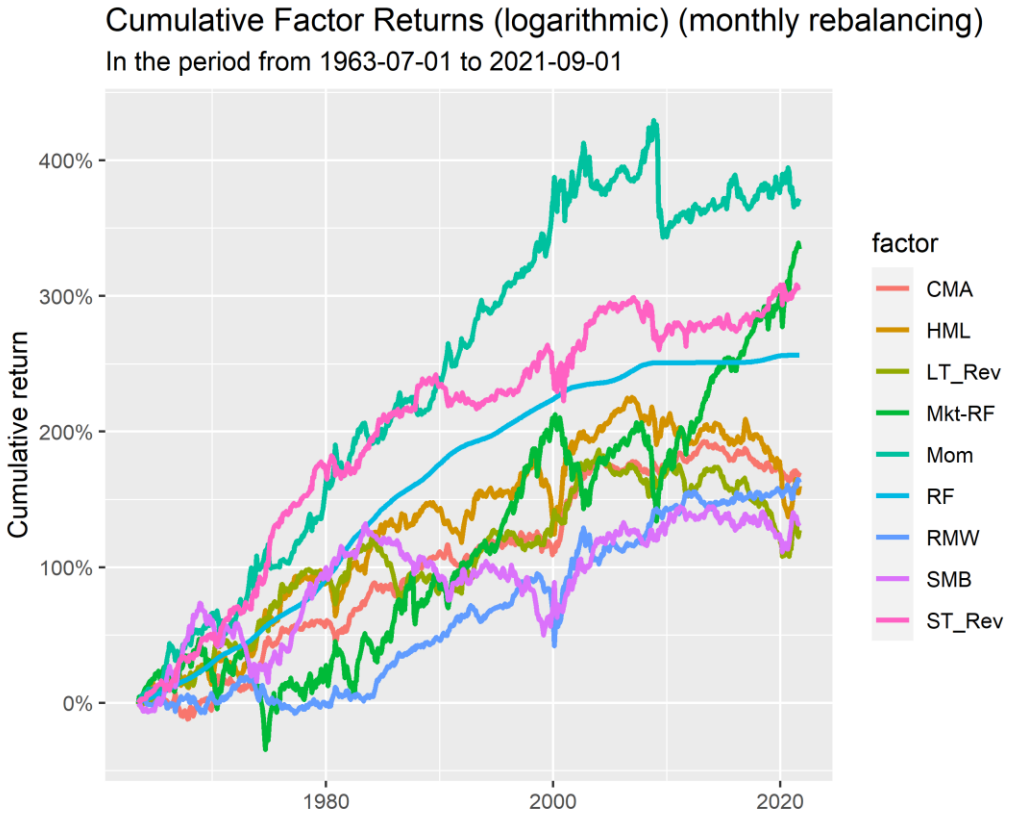
```
# Load factors (daily)
ff_5_factors <- download_french_data("Fama/French 5 Factors (2x3)")
mom_factors <- download_french_data("Momentum Factor (Mom)")
strev_factors <- download_french_data("Short-Term Reversal Factor (ST Rev)")
ltrev_factors <- download_french_data("Long-Term Reversal Factor (LT Rev)")
```

- And adjust the date format conversion:

```
# Convert date to date format
factors2 <- factors2 %>%
   mutate(date = as.Date(paste0(as.character(factors$date), '01'), format='%Y%m%d'))
```

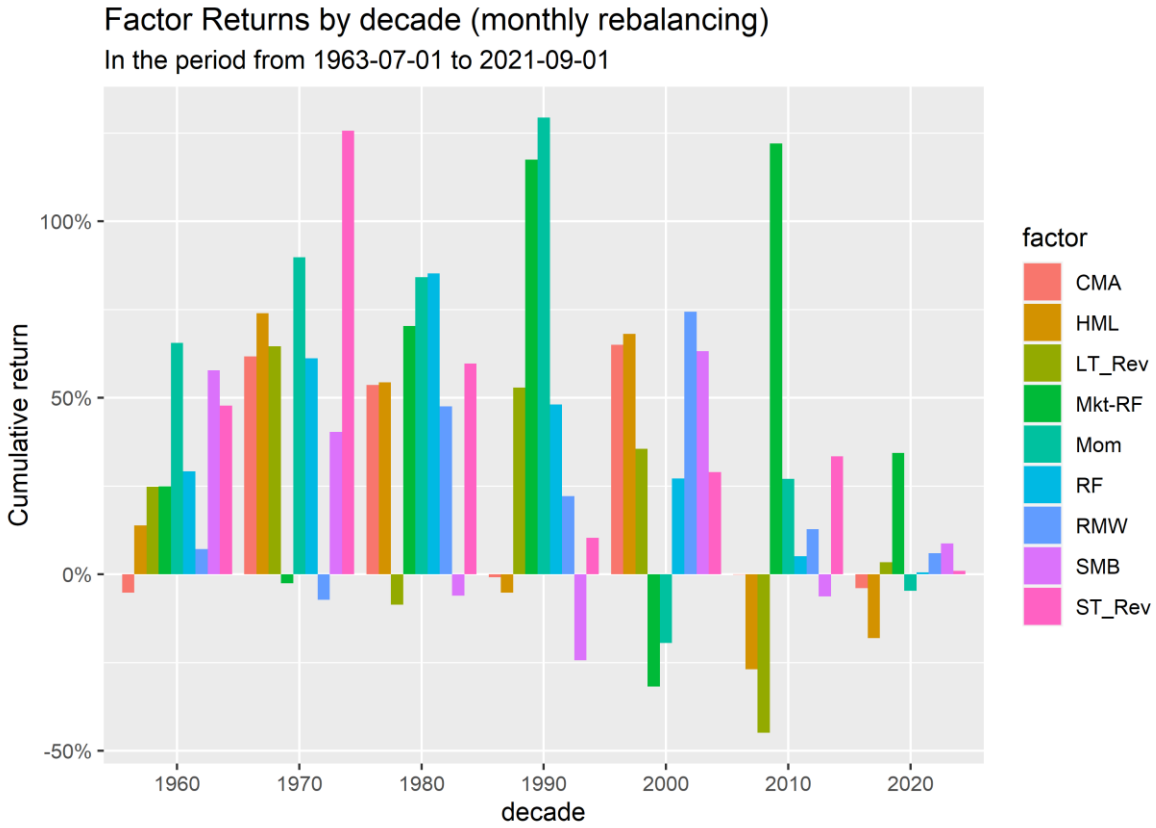
Factor-Mimicking portfolio returns





- Once we moved to the more realistic **monthly rebalancing**, the returns of the **ST_Rev** factor dropped significantly, but still stay relativelly high
- The cumulativelly most profitable factor is now the **momentum factor (Mom)**, although even its returns are negative in the last two decades, due to its large losses during the dot.com bubble (2000), the global financial crisis (2008), and the Covid crisis (2020)

Factor-Mimicking portfolio returns



- The worsening performance over the last two decades is apparent for the monthly-rebalanced factor mimicking portfolios as well (as expected)

Portfolio optimization and CAPM

- Multi-Factor models
- Fama-French data library
- **Multi-Factor models in practice**

Factor models and MPT

- An estimated factor model can be used for portfolio optimization in the MPT framework:
  1. Start with an estimated factor model: $R_{i,t} = \alpha_i + \beta_{i,1}F_{1,t} + \cdots + \beta_{i,n}F_{n,t} + \varepsilon_{i,t}$
  2. This can be written in matrix notation as: $\boldsymbol{R}_t = \boldsymbol{A} + \boldsymbol{B}\boldsymbol{F}_t + \boldsymbol{E}_t$
     where $\boldsymbol{A}$ is *Nx1* vector of historical alphas, $\boldsymbol{B}$ is *NxK* matrix of factor loadings, $\boldsymbol{F}_t$ is *Nx1* vector of factor returns and $\boldsymbol{E}_t$ is *Nx1* vector of residuals
  3. Expected returns can then be eastimated as: $\boldsymbol{\mu} = E(\boldsymbol{R}_{t+1}) = r_f + \boldsymbol{B}E(\boldsymbol{F}_{t+1})$
     where $E(\boldsymbol{F}_{t+1})$ can be estimated either with a naive forecast $E(\boldsymbol{F}_{t+1}) = \overline{\boldsymbol{F}}_{t-n+1:t}$ (i.e. with an average over the last n periods), or with a suitable time-series model (VAR, HMM, RNN, etc.)
  4. Covariance matrix of asset returns $\boldsymbol{\Sigma}$ is then: $\boldsymbol{\Sigma} = \boldsymbol{B}\boldsymbol{\Omega}\boldsymbol{B}' + \boldsymbol{D}$
     where $\boldsymbol{D}$ is the covariance matrix of residuals $\boldsymbol{E}_t$, which is usually assumed to be diagonal, and $\boldsymbol{\Omega}$ is the covariance matrix of factor returns, estimated either as $\boldsymbol{\Omega} = \mathrm{cov}(\boldsymbol{F}_{t-n+1:t})$, or with a suitable dynamic correlation model (such as DCC-GARCH)
  5. Expected return of a portfolio $\mu_{\boldsymbol{w}}$ is then: $\mu_{\boldsymbol{w}} = \boldsymbol{w}'\boldsymbol{\mu}$ (for any asset weights $\boldsymbol{w}$)
  6. And portfolio return volatility $\sigma_{\boldsymbol{w}}$ is: $\sigma_{\boldsymbol{w}} = \sqrt{\boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w}}$ (for any asset weights $\boldsymbol{w}$)
  7. The optimal portfolio can thus be found as: $\widehat{\boldsymbol{w}} = \max_{\boldsymbol{w}} \frac{\mu_{\boldsymbol{w}} - r_f}{\sigma_{\boldsymbol{w}}}$

Factor models & portfolio optimization

- **Task:** Use the **Mkt-RF, HML, SMB, RWA, CMA, Mom, ST-Rev** and **LT-Rev** factors from the Fama-French library (since 1.1.1990) to construct a factor model of DJIA stock returns in the period from 1.1.1990 - 31.12.2015, and use it to perform portfolio optimization based on the derived values of $\mu_{\boldsymbol{w}}$ and $\sigma_{\boldsymbol{w}}$
- **Guide:**
  1. Download the daily factor returns data from the Fama-French library
  2. Download the DJIA stock price data in the period since 1.1.1990
  3. Estimate a factor model separately for each stock to get: $\boldsymbol{R}_t = \boldsymbol{A} + \boldsymbol{B}\boldsymbol{F}_t + \boldsymbol{E}_t$
  4. Estimate vector of expected stock returns: $\boldsymbol{\mu} = r_f + \boldsymbol{B}E(\boldsymbol{F}_{t+1}) \cong r_f + \boldsymbol{B}\bar{\boldsymbol{F}}_{0:t}$ where $\bar{\boldsymbol{F}}_{0:t}$ denotes the average historical factor returns and r_f the risk-free rate
  5. Estime the covariance matrix of stock returns $\boldsymbol{\Sigma}$ with: $\boldsymbol{\Sigma} = \boldsymbol{B}\boldsymbol{\Omega}\boldsymbol{B}' + \boldsymbol{D}$ where $\boldsymbol{D}$ is the covariance matrix of residuals $\boldsymbol{E}_t$, assumed to be diagonal, and $\boldsymbol{\Omega}$ is the historical covariance matrix of factor returns
  6. Write function to compute portfolio Sharpe ratio $Sharpe_{\boldsymbol{w}} = \max_{\boldsymbol{w}} \frac{\mu_{\boldsymbol{w}} - r_f}{\sigma_{\boldsymbol{w}}}$ where $\mu_{\boldsymbol{w}} = \boldsymbol{w}'\boldsymbol{\mu}$ is the expected portfolio return, $\sigma_{\boldsymbol{w}} = \sqrt{\boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w}}$ is the volatility of portfolio returns, and $\boldsymbol{w}$ is a vector of portfolio weights
  7. Find the optimal portfolio $\hat{\boldsymbol{w}} = \max_{\boldsymbol{w}} \frac{\mu_{\boldsymbol{w}} - r_f}{\sigma_{\boldsymbol{w}}}$, corresponding to the **200/100 strategy**, i.e. using the constrains that $\sum \boldsymbol{w} = 1$ and $\sum \boldsymbol{w} I(\boldsymbol{w} > 0) = 2$
  8. Compute the portfolio returns in the out-sample period (2016-2021)

Step 1: Factor data download (1)

1) Download the daily factor returns data from the Fama-French library

Load relevant libraries:

```
# Load libraries
library(tidyverse)
library(tidyquant)
library(frenchdata)
library(broom)
```

Factor data download:

```
# Load factors (daily)
ff_5_factors <- download_french_data("Fama/French 5 Factors (2x3) [Daily]")
mom_factors <- download_french_data("Momentum Factor (Mom) [Daily]")
strev_factors <- download_french_data("Short-Term Reversal Factor (ST Rev) [Daily]")
ltrev_factors <- download_french_data("Long-Term Reversal Factor (LT Rev) [Daily]")
# Extract factors
factors_ff5 <- ff_5_factors$subsets$data[[1]]
factors_mom <- mom_factors$subsets$data[[1]]
factors_strev <- strev_factors$subsets$data[[1]]
factors_ltrev <- ltrev_factors$subsets$data[[1]]
# Join factors from different datasets
factors <- factors_ff5 %>%
  left_join(factors_mom,by="date") %>%
  left_join(factors_strev,by="date") %>%
  left_join(factors_ltrev,by="date")
```

Step 1: Factor data download (2)

1) Download the daily factor returns data from the Fama-French library

Convert returns to percent (expressed as decimal):

```
# Convert returns to percent
as_percent = function(x) {x/100}
factors2 <- factors %>%
  mutate_at(vars(-date),as_percent)
```

Convert date in YYYYMMDD format to date format:

```
# Convert returns # Convert date to date format
factors2 <- factors2 %>%
  mutate(date = as.Date(as.character(date), "%Y%m%d"))
```

Convert compound returns to log-returns (for consistency):

```
# Convert compound returns to log-returns
as_logreturns = function(x) {log(1+x)}
factors3 <- factors2 %>%
  mutate_at(vars(-date),as_logreturns)
```

Rename the Mkt-RF factor to Mkt_RF (so that R does not interpret „-“ as minus):

```
# Rename the Mkt-RF factor
factors3 <- factors3 %>%
  rename(Mkt_RF = c("Mkt-RF"))
```

Step 2: DJIA data download (1)

2) Download the DJIA stock price data in the period since 1.1.1990

DJIA data download:

```
# Get DJIA components
info <- tq_index("DOW")
# Download settings:
tStart <- "1990-01-01"
tEnd   <- today()
# Load DJIA stock prices (+ include info about each stock)
prices <- info %>% tq_get(get = "stock.prices", from = tStart, to = tEnd)
```

We then compute the log-returns of all DJIA stocks:

```
# Compute daily log-returns
prices <- prices %>%
  group_by(symbol) %>%
  mutate(logprice = log(adjusted),
         return = logprice - lag(logprice),
         return = ifelse(is.na(return),0,return))
```

And remove the stocks for which the whole history is not available:

```
# Remove stocks that were not available since the first day of the sample
minDate <- min(prices$date)
prices2 <- prices %>%
  group_by(symbol) %>%
  mutate(mindate = min(date)) %>%
  filter(mindate==minDate)
```

Step 2: DJIA data download (2)

2) Download the DJIA stock price data in the period since 1.1.1990

Finally, we join the factor returns dataset and the DJIA returns dataset together:

```
# Select in-sample
sYear <- 2015
# Join the datasets
prices3 <- prices2 %>%
  left_join(factors3,by="date")
# Fill the missing values
prices3 <- prices3 %>%
  fill(.direction = c("down"))
# Variable names
fnames <- c(variable.names(factors3),"symbol","return")
# Convert to wide format
wprices <- prices3 %>%
  select(all_of(fnames)) %>%
  spread(symbol,return)
# Select in-sample
wpricesIn <- wprices %>%
  filter(year(date) <= sYear)
# Convert into matrix
XF <- as.matrix(wprices[,11:ncol(wprices)])
X <- as.matrix(wpricesIn[,11:ncol(wpricesIn)])
```

Step 3: Estimate the factor model (1)

3) Estimate a factor model separately for each stock to get: $R_t = A + BF_t + E_t$

```
# Select factor names
cFactors <- variable.names(factors3)[-1]
cFactors <- cFactors[cFactors!="RF"]
# Write regression formula
form <- paste0("exc_return ~ ")
i <- 1
for (cf in cFactors) {
  if (i==1) {
    form <- paste0(form, cf)
    i <- 0
  } else {
    form <- paste0(form, " + ", cf)
  }
}
# Estimate the regression models for each stock
fit <- prices3 %>%
  group_by(symbol) %>%
  filter(year(date) <= sYear) %>%
  mutate(exc_return = return - RF) %>%
  select(all_of(c("symbol","date","exc_return",cFactors))) %>%
  nest(data = -symbol) %>%
  mutate(
    fit = map(data, ~ lm(as.formula(form), data = .x)),
    tidied = map(fit, tidy)
  )
```

Step 3: Estimate the factor model (2)

3) Estimate a factor model separately for each stock to get: $R_t = A + BF_t + E_t$

We can then extract the relevant results:

```
# Extract results
fit <- fit %>%
  select(symbol,tidied) %>%
  unnest(tidied)
# Convert into a wide format
coefs <- fit %>%
  select(symbol,term,estimate) %>%
  spread(term, estimate) %>%
  select(all_of(c("symbol","(Intercept)",cFactors)))
```

Telling us the sensitivities of DJIA stock returns to the 8 risk factors:

	symbol	(Intercept)	Mkt_RF	SMB	HML	RMW	CMA	Mom	ST_Rev	LT_Rev
1	AAPL	5.782164e-04	0.9992527	-0.03462973	-0.497468567	-0.15309974	-1.27489476	-0.044819405	-0.076522228	0.373049263
2	AMGN	5.043184e-04	0.9384588	-0.18656452	-0.696324919	-0.07331741	0.08978566	0.136626116	-0.038542817	0.049472534
3	AXP	4.532501e-05	1.3969738	-0.25858586	0.631143006	0.17065757	-0.18595780	-0.231732609	-0.113135841	-0.125860952
4	BA	-1.151163e-04	1.0342833	0.05919195	-0.050221029	0.37493786	0.40985260	-0.048290498	0.038744268	0.008960038
5	CAT	-7.915694e-05	1.1855017	0.21631051	0.090360840	0.53655397	0.45619187	-0.203846025	0.015603599	-0.076333446

We can see that **AAPL** exhibits highest **positive sensitivity** to the **MkT_RF** factor (overal market movements, and the highest **negative sensitivity** to the **CMA** factor (returns of firms investing conservativelly minus return of firms investing aggresivelly). This is to be expected, as AAPL is a technological company that invests aggresivelly (into innovations, etc.). Market conditions that favour conservativelly-investing firms and disfavour aggresivelly-investing ones would thus be negative for AAPL, which is captured by its negative sensitivity to the CMA factor.

Step 4: Estimate expected returns

4) Estimate vector of expected stock returns: $\boldsymbol{\mu} = r_f + \boldsymbol{BE}(\boldsymbol{F_{t+1}}) \cong r_f + \boldsymbol{B}\overline{\boldsymbol{F}}_{\boldsymbol{0:t}}$

We first compute the historical average factor returns $\overline{\boldsymbol{F}}_{\boldsymbol{0:t}}$:

```
# Compute historical factor returns
freturns <- wprices %>%
  select(all_of(cFactors))
freturnsIn <- wpricesIn %>%
  select(all_of(cFactors))
# Convert to matrix
FX <- as.matrix(freturnsIn)
FXF <- as.matrix(freturns)
# Average historical factor returns
FM <- colMeans(FX)
```

We can then compute the expected stock returns with $\boldsymbol{\mu} = r_f + \boldsymbol{B}\overline{\boldsymbol{F}}_{\boldsymbol{0:t}}$:

```
# Risk-free interest rate (for the out-sample period)
rf <- 0.01
# Factor sensitivities in matrix form
phi <- as.matrix(coefs[,-1])
row.names(phi) <- coefs$symbol
# Vector of expected returns (excluding alpha)
M <- as.vector(phi  %*% c(1,FM))
M <- (rf/252) + as.vector(phi[,-1]  %*% FM)
names(M) <- coefs$symbol
```

Step 5: Estimate covariance matrix (1)

5) Estimate the covariance matrix of stock returns $\boldsymbol{\Sigma}$ with: $\boldsymbol{\Sigma = B\Omega B' + D}$

We first compute the covariance matrix of factor returns $\boldsymbol{\Omega}$:

```
# Compute correlation and covariance matrix of factor returns
RF <- cor(FX) # Correlation matrix of factor returns
SF <- cov(FX) # Covariance matrix of factor returns
```

Then the covariance matrix of model residuals $\boldsymbol{D}$:

```
# Compute correlation and covariance matrix of residuals
X_Pred <- cbind(1,FX) %*% t(phi)
X_Res <- X - X_Pred
RR <- cor(X_Res)   # Correlation matrix of residuals
SR <- cov(X_Res)   # Covariance matrix of residuals
```

We can then compute the covariance matrix of stock returns $\boldsymbol{\Sigma}$ with:

```
# Compute covariance matrix of stock returns
S <- phi[,-1] %*% SF %*% t(phi[,-1]) + diag(diag(SR))
```

Note: According to the assumptions of the factor model, all systematic dependencies between individual stock returns should be captured through their correlations with the systematic market factors, allowing us to omitt all non-diagonal elements of $\boldsymbol{D}$

We will verify this assumption on the next slide

Step 5: Estimate covariance matrix (2)

5) Estimate the covariance matrix of stock returns $\boldsymbol{\Sigma}$ with: $\boldsymbol{\Sigma = B\Omega B' + D}$

To verify the assumption that all non-diagonal elements of $\boldsymbol{D}$ could be omitted, we will also compute the historical correlation matrix of stock returns:

```
# Compute correlation and covariance matrix
RH <- cor(X)   # Correlation matrix
SH <- cov(X)   # Covariance matrix
```

We can then compare the:

a) Correlation matrix of historical stock returns
b) Correlation matrix of the factor model residuals

The comparison is performed for the first 5 stocks:

a) Correlation matrix of returns:

	AAPL	AMGN	AXP	BA	CAT
AAPL	1.0000000	0.2226036	0.2654051	0.2138250	0.2665176
AMGN	0.2226036	1.0000000	0.2874062	0.2518754	0.2112634
AXP	0.2654051	0.2874062	1.0000000	0.4070307	0.4461676
BA	0.2138250	0.2518754	0.4070307	1.0000000	0.3847919
CAT	0.2665176	0.2112634	0.4461676	0.3847919	1.0000000

b) Correlation matrix of residuals:

	AAPL	AMGN	AXP	BA	CAT
AAPL	1.000000000	-0.011052477	-3.871927e-02	-0.023976249	3.855162e-02
AMGN	-0.011052477	1.000000000	1.087739e-03	0.001836119	-6.302564e-02
AXP	-0.038719272	0.001087739	1.000000e+00	0.012992233	3.291450e-03
BA	-0.023976249	0.001836119	1.299223e-02	1.000000000	4.577848e-02
CAT	0.038551619	-0.063025642	3.291450e-03	0.045778476	1.000000e+00

We can see that while there was non-negligible correlation (0.2-0.4) present among the original stock returns, most of it got captured by the systematic factors of our model, with only negligible correlation (0.01-0.04) remaining among the model residuals

Step 6: Compute the Sharpe ratio

6) Write function to compute portfolio Sharpe ratio $Sharpe_{\boldsymbol{w}} = \max_{\boldsymbol{w}} \frac{\mu_{\boldsymbol{w}} - r_f}{\sigma_{\boldsymbol{w}}}$

We first define the inputs used in the function:

```
# Define the inputs (annualized)
mju <- M * 252
sigma <- S * 252
r <- rf
# Initial weights
w0 <- rep(1/nrow(S),nrow(S))
```

And then write a function to compute the Sharpe ratio of the portfolio:

```
# Sharpe ratio function
mysharpe = function(w,mju,sigma,r) {

  # Compute portfolio expected return and volatility
  mw <- as.vector(t(w) %*% mju)
  sw <- as.vector((t(w) %*% sigma %*% w)^(1/2))

  # Compute Sharpe ratio
  Sharpe <- (mw-r)/sw

  # Return Sharpe
  return(Sharpe)

}
```

Step 7: Compute the Sharpe ratio

7) Find the optimal portfolio $\hat{w} = \max_{w} \frac{\mu_w - r_f}{\sigma_w}$ corresponding to the 200/100 strategy

First, we will write the objective function for the 200/100 strategy portfolio, i.e. using the constrains that $\sum w = 1$ and $\sum w I(w > 0) = 2$

```
# OBJECTIVE FUNCTION - 200/100 - L-BFGS-B
objective = function(w) {
  # Rescale the weights
  wn <- w/sum(abs(w))
  # Set minimization objective
  obj <- -mysharpe(wn,mju,sigma,r)+100*((sum(w)-1)^2)+100*((sum(w*(w>0))-2)^2)
  # Get oputput
  return(obj)
}
# Weight bounds (LONG-SHORT, 200/100)
wmin <- numeric(length(w0))-1
wmax <- numeric(length(w0))+1
```

We can then **optimize the portfolio weights** with:

```
# Optimize the portfolio weights
optRes <- optim(w0, objective, method = "L-BFGS-B", lower = wmin, upper = wmax)
```

And extract the optimized weights with:

```
# Get weights
we <- optRes$par
names(we) <- names(M)
```

Step 8: Compute out-sample returns (1)

8) Compute the portfolio returns in the out-sample period (2016-2021)

We first compute the return vectors and gather the data:

```
# Compute portfolio returns (full-sample)
R0 <- XF %*% w0  # Equally weighted benchmark
RE <- XF %*% we  # Optimized portfolio
# Join into a tibble
portfolio <- tibble("date" = wprices$date,
                    "benchmark" = R0,
                    "optimised" = RE)
# Gather
portfolio <- portfolio %>%
  gather(method,return,benchmark:optimised)
```

The **full-sample logarithmic returns** can then be plotted with:

```
# Plot on chart - Logarithmic returns - Full-sample
portfolio %>%
  group_by(method) %>%
  mutate(cumreturn = cumsum(return)) %>%
  ggplot(aes(x = date, y = cumreturn, color = method)) +
  geom_line(size = 1) +
  labs(title = "Cumulative Returns - Logarithmic (Full-Sample)",
       subtitle = paste0("In the period from ",year(tStart)," to ",year(tEnd)),
       x = "", y = "Cumulative returns") +
  scale_y_continuous(labels = scales::percent)
```

Step 8: Compute out-sample returns (2)

8) Compute the portfolio returns in the out-sample period (2016-2021)

The **out-sample logarithmic returns** can be plotted with:

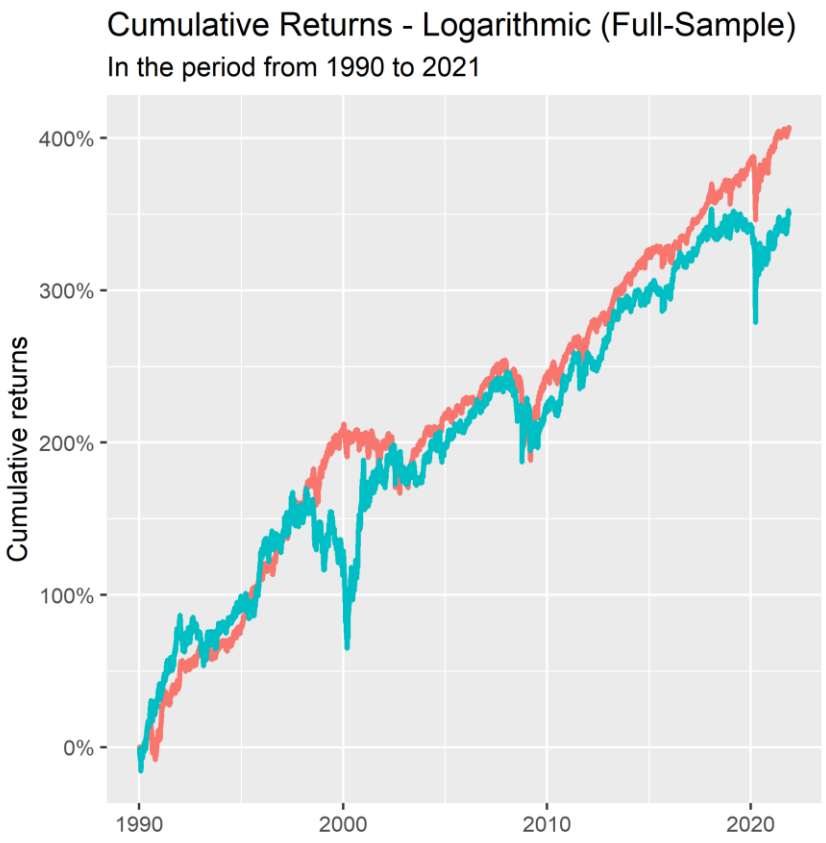
```
# Plot on chart - Logarithmic returns - Out-sample
portfolio %>%
  filter(year(date)>sYear) %>%
  group_by(method) %>%
  mutate(cumreturn = cumsum(return)) %>%
  ggplot(aes(x = date, y = cumreturn, color = method)) +
  geom_line(size = 1) +
  labs(title = "Cumulative Returns - Logarithmic (Out-Sample)",
       subtitle = paste0("In the period from ",sYear," to ",year(tEnd)),
       x = "", y = "Cumulative returns") +
  scale_y_continuous(labels = scales::percent)
```

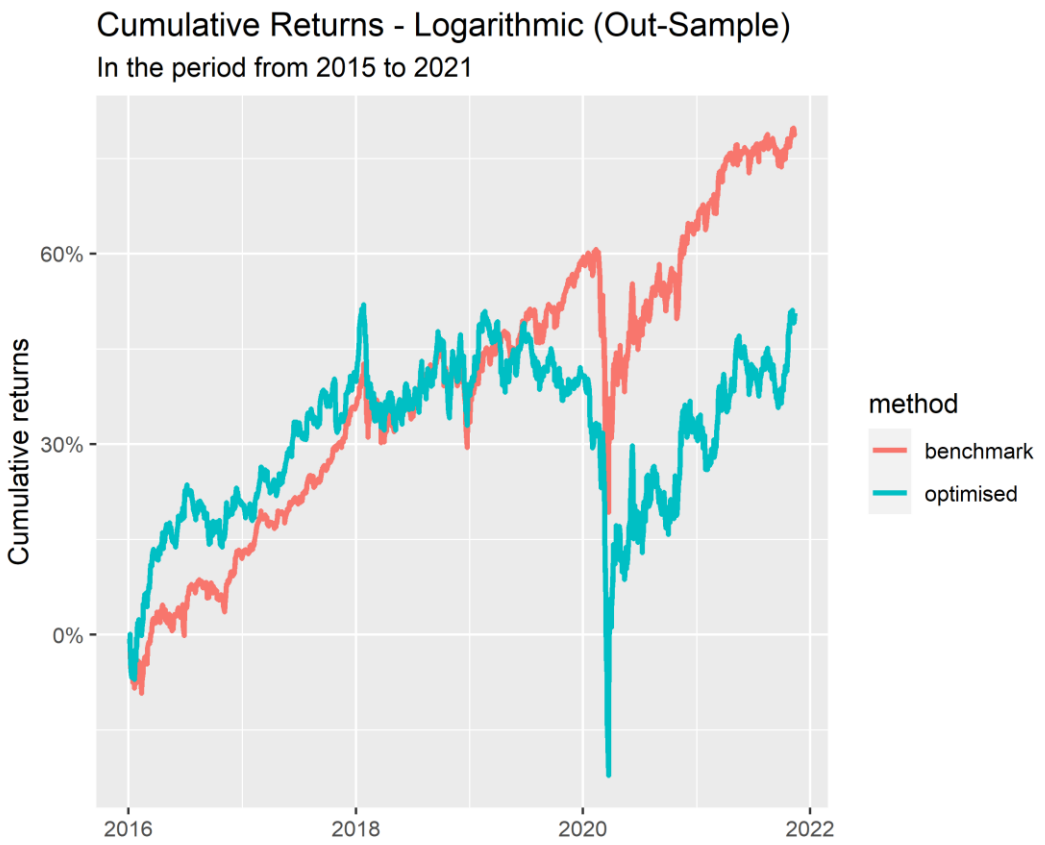
The **out-sample exponential returns** with:

```
# Plot on chart - Exponential returns - Out-sample
portfolio %>%
  filter(year(date)>sYear) %>%
  group_by(method) %>%
  mutate(cumreturn = cumsum(return)) %>%
  mutate(cumreturn = exp(cumreturn)-1) %>%
  ggplot(aes(x = date, y = cumreturn, color = method)) +
  geom_line(size = 1) +
  labs(title = "Cumulative Returns - Exponential (Out-Sample)",
       subtitle = paste0("In the period from ",sYear," to ",year(tEnd)),
       x = "", y = "Cumulative returns") +
  scale_y_continuous(labels = scales::percent)
```

Step 8: Compute out-sample returns (3)

Full-sample vs. out-sample logarithmic returns:

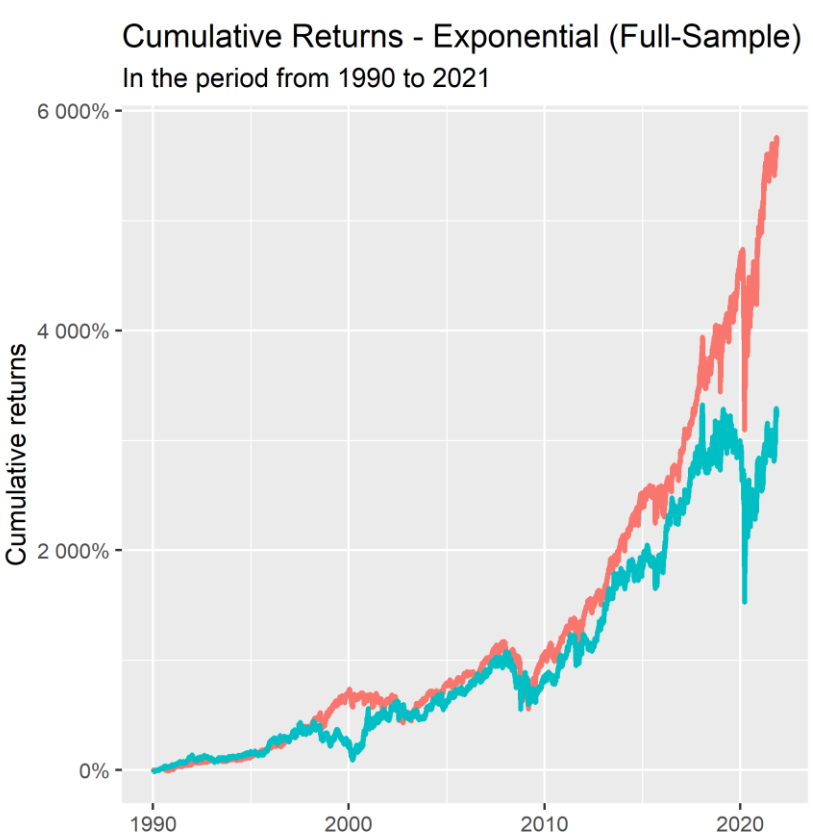


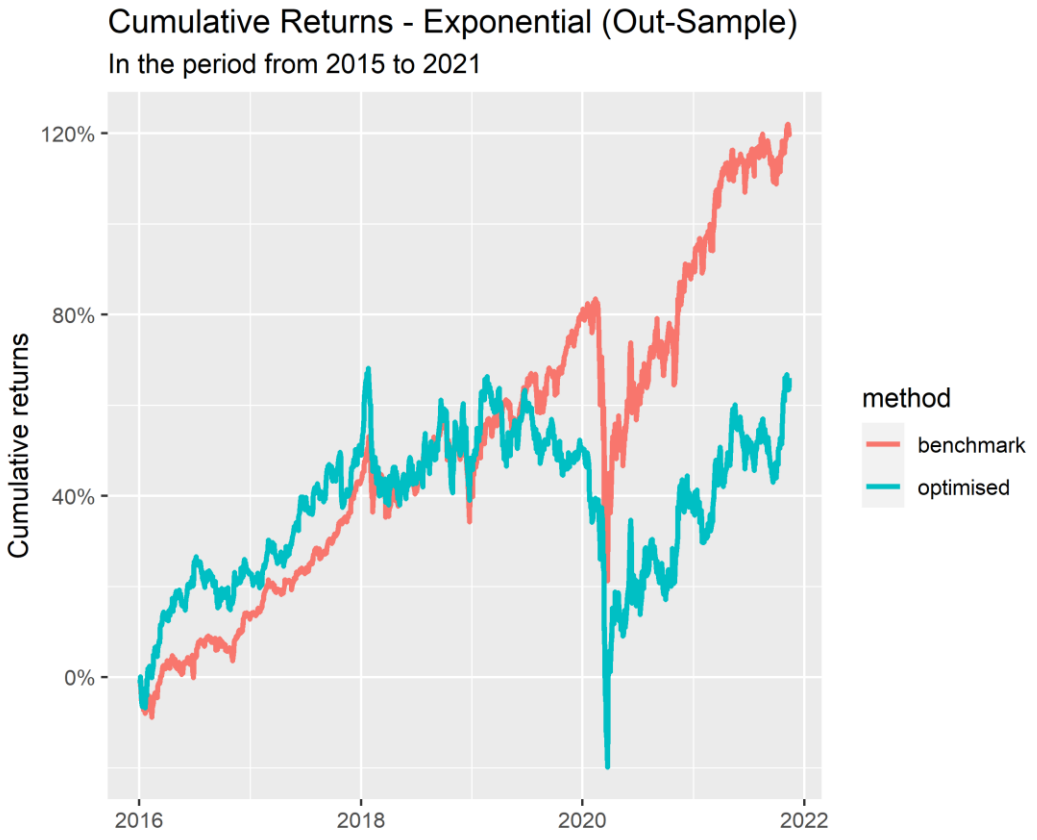


We can see that the model clearly underperforms the benchmark (on in-sample as well as out-sample) → Factor sensitivities were not associated with above-average returns for the DJIA stocks in the period from 1990-2021

Step 8: Compute out-sample returns (3)

Full-sample vs. out-sample exponential returns:





We can see that the model clearly underperforms the benchmark (on in-sample as well as out-sample) → Factor sensitivities were not associated with above-average returns for the DJIA stocks in the period from 1990-2021

Fama-Macbeth regression (in-sample)

To see why the model does not seem to work so well, we can check the **statistical signifikance** of the relationship between the **factor sensitivities** and the **historical stock returns** (so called Fama-Macbeth regression)

Fama-Macbeth regression (In-Sample):

```
# Fama-Macbeth regression (in-sample)
MH <- colMeans(X)
check1 <- lm(MH ~ phi[,-1])
summary(check1)
```

```
Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)      0.03036    0.05455   0.557  0.58552
phi[, -1]Mkt_RF  0.11796    0.05201   2.268  0.03754 *
phi[, -1]SMB     0.07026    0.03142   2.236  0.03992 *
phi[, -1]HML    -0.02456    0.01679  -1.462  0.16299
phi[, -1]RMW     0.01234    0.02305   0.535  0.59988
phi[, -1]CMA    -0.02769    0.02187  -1.266  0.22358
phi[, -1]Mom     0.26927    0.06865   3.923  0.00121 **
phi[, -1]ST_Rev  0.08850    0.07849   1.127  0.27616
phi[, -1]LT_Rev  0.06213    0.05711   1.088  0.29277
```

Note: With the exception of the **Mom** factor, and to a lesser degree the **Mkt_RF** factor and **SMB** factor, the relationships between factor sensitivities and historical stock returns are insignificant

Take note that a 0.26927 sensitivity to **Mom** indicates that stocks with 100% sensitivity (phiMom=1) to the Momentum factor would provide a risk premium of 26.927% p.a.

Fama-Macbeth regression (out-sample)

Fama-Macbeth regression (Out-Sample):

```
# Out-sample stock returns
wpricesOut <- wprices %>%
  filter(year(date) <= sYear)
XOut <- as.matrix(wpricesOut[,11:ncol(wpricesOut)])
MHOut <- colMeans(XOut)
check2 <- lm(MH ~ phi[,-1])
summary(check2)
```

```
Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)      0.09486    0.15941   0.595  0.56012
phi[, -1]Mkt_RF  0.02471    0.15200   0.163  0.87288
phi[, -1]SMB     0.02461    0.09180   0.268  0.79204
phi[, -1]HML    -0.04743    0.04907  -0.967  0.34815
phi[, -1]RMW     0.18403    0.06736   2.732  0.01477 *
phi[, -1]CMA    -0.22687    0.06390  -3.550  0.00266 **
phi[, -1]Mom    -0.18568    0.20060  -0.926  0.36841
phi[, -1]ST_Rev  0.30238    0.22937   1.318  0.20596
phi[, -1]LT_Rev -0.32920    0.16690  -1.972  0.06610 .
```

Note: We can see that in the out-sample, the most significant factor becomes **CMA** (but with negative coefficient, contrary to the Fama-French assumptions), followed by **RMW**

Additionally, the coefficient of the **Mom** factor (which was most significant in the in-sample), becomes negative (although statistically insignificant), indicating that DJIA stock sensitivity to the momentum factor was associated with negative risk premium in the out-sample (2016-2021)

Step 4: modification (1)

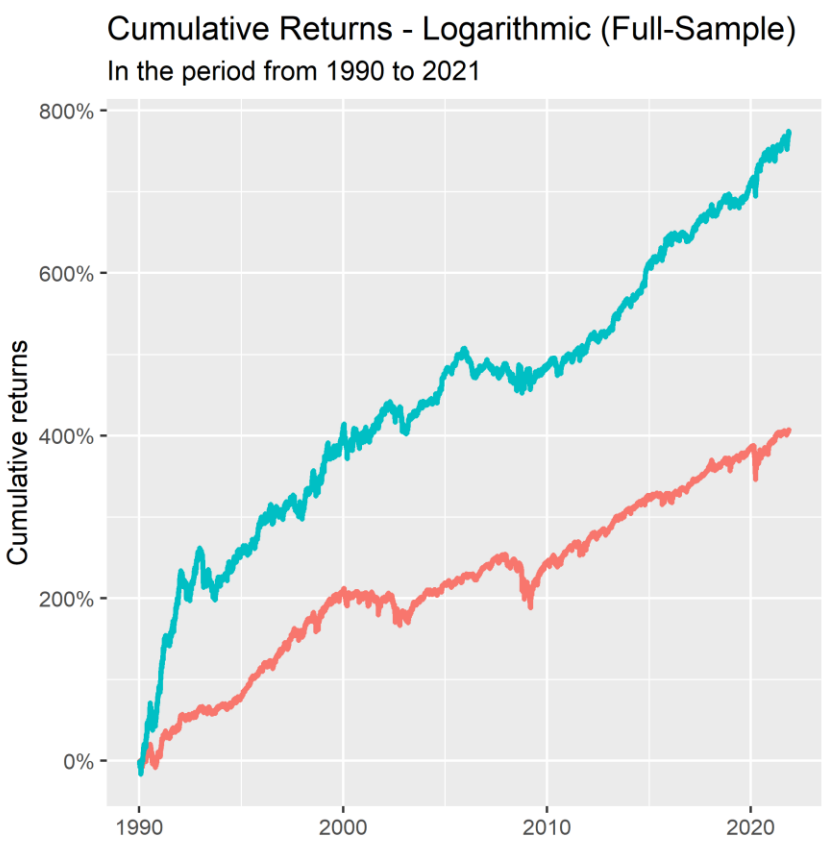
Alternatively, **contrary to the Fama-French assumption** that expected returns depend only on risk-factor sensitivities, we can decide to keep historical alphas in the model and estimate expected stock returns with: $\boldsymbol{\mu = A + B\bar{F}_{0:t}}$:

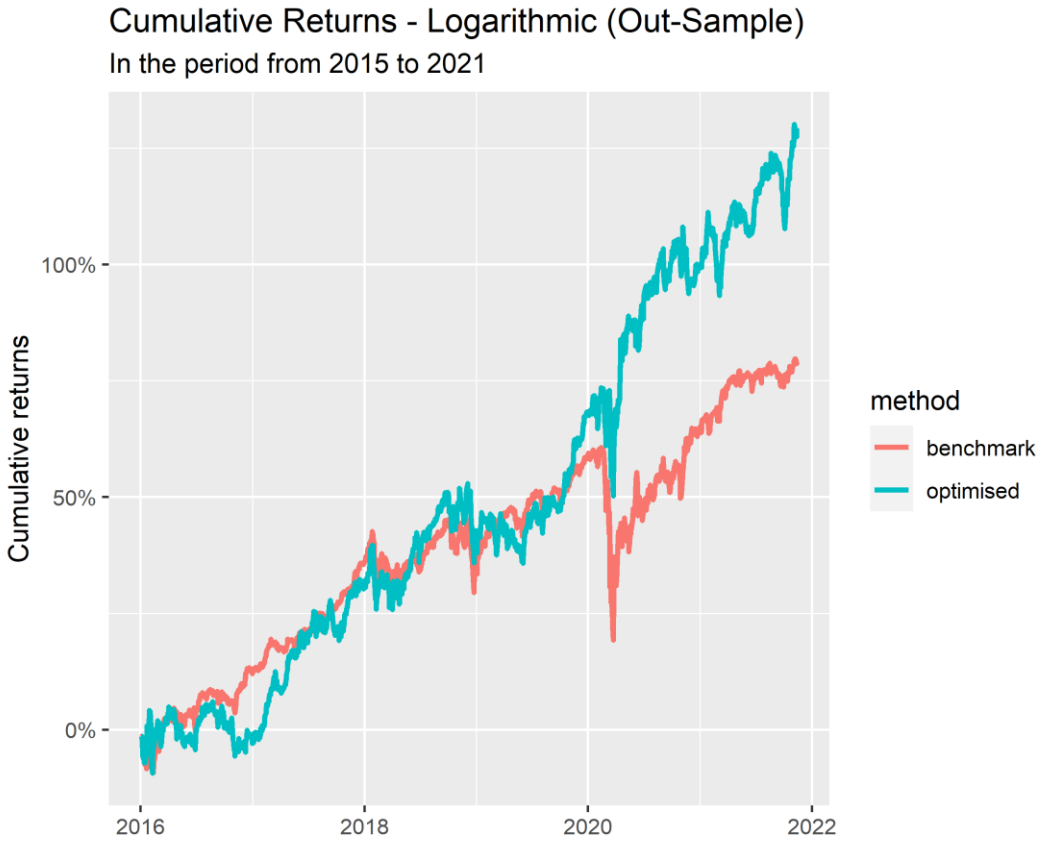
```
# Vector of expected returns (including alpha)
M <- as.vector(phi  %*% c(1,FM))
names(M) <- coefs$symbol
```

Take note that this is **contrary to the efficient market hypothesis**, as it would indicate a belief that stocks that out-performed in the past (over 1990-2015), will out-perform also in the future (2016-2021)

Step 4 modification (2)

Full-sample vs. out-sample logarithmic returns:

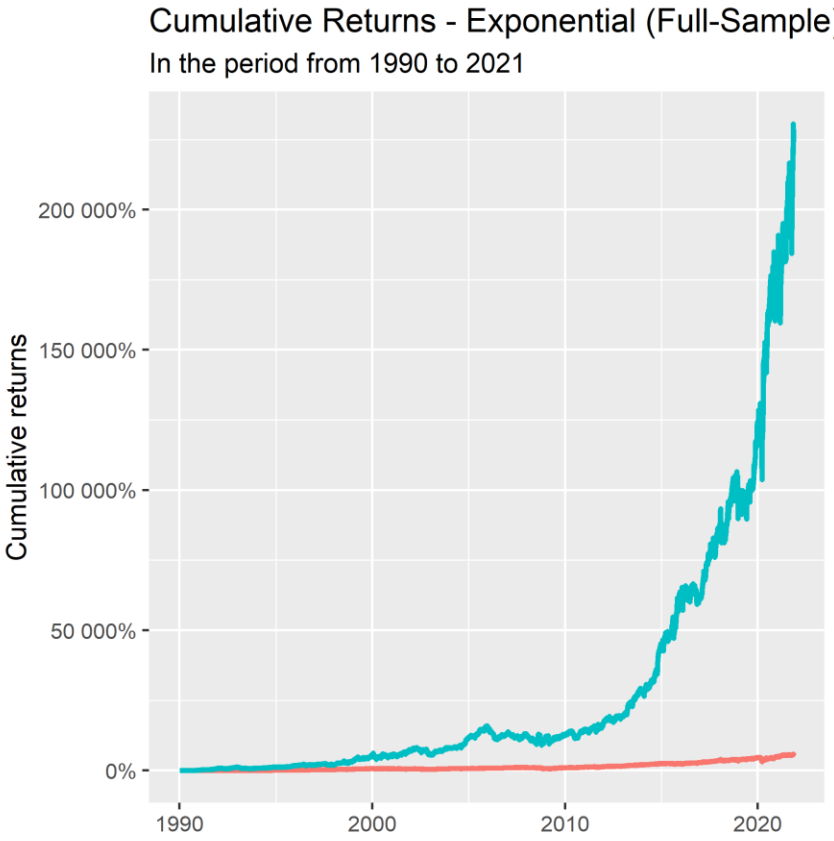


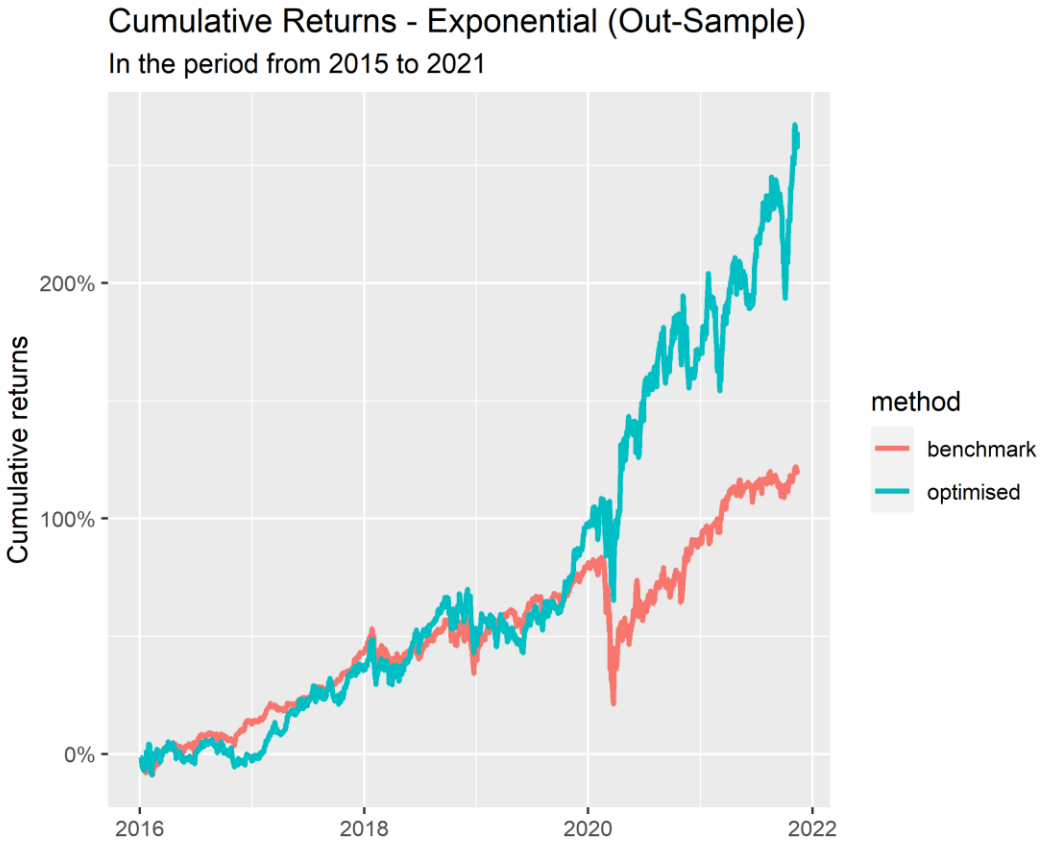


We can see that the results look better than for the original model. On the in-sample it is to be expected, as we used the historical alphas (out-performances) in the optimization. In the out-sample the result might be caused either by (1) chance, (2) survivorship bias of the dataset, or possibly (3) presence of a certain ultra-long-term momentum effect in the DJIA stocks

Step 4 modification (3)

Full-sample vs. out-sample exponential returns:





We can see that the model clearly underperforms the benchmark (on in-sample as well as out-sample) → Factor sensitivities were not associated with above-average returns for the DJIA stocks in the period from 1990-2021